

Designing Verification Checkpoints to Mitigate Silent Errors in Agentic AI Pipelines

Scholarly Research Paper

Abstract

Agentic artificial intelligence (AI) systems extend large language models (LLMs) from passive text generation to autonomous or semi-autonomous workflows involving planning, tool invocation, information retrieval, code execution, memory, and interaction with external environments. This increased agency creates a distinctive reliability problem: silent errors can occur at intermediate stages without producing an explicit exception, yet subsequently propagate through the pipeline and lead to incorrect final decisions or actions. Existing evaluation approaches frequently emphasize end-to-end task success, which can obscure the location, type, and causal structure of intermediate failures. This paper examines the design of verification checkpoints as an architectural mechanism for detecting and containing such failures. Drawing on research on reasoning traces, process supervision, tool-using agents, hallucination detection, retrieval-augmented generation, agent evaluation, and LLM-based judging, the paper develops a checkpoint-oriented reliability framework. The proposed framework separates verification into five dimensions: input validation, plan verification, tool-call verification, state-transition verification, and output verification. It further introduces adaptive checkpoint placement based on risk, uncertainty, reversibility, and downstream error propagation. The paper argues that verification should not be treated solely as a final quality-control layer; instead, it should be distributed throughout an agentic pipeline and coupled with rollback, retry, escalation, and recovery mechanisms. Current approaches demonstrate promising components but lack standardized metrics for checkpoint effectiveness, false-negative rates, verification cost, and error containment. Future research should therefore investigate causal fault localization, calibrated verification, adversarial checkpoint testing, cross-agent verification, and benchmark suites specifically designed to measure silent-error propagation.

Keywords: Agentic AI, large language models, AI agents, verification, silent errors, reliability, process supervision, tool use, hallucination, fault tolerance, checkpointing, autonomous systems

1. Introduction

Large language models have evolved from systems that primarily generate text into components capable of interacting with tools, executing multi-step plans, accessing external information, writing and executing code, and coordinating with other agents. Frameworks such as ReAct demonstrate how reasoning and acting can be interleaved, while Toolformer demonstrates that language models can learn when and how to invoke external tools [1], [2]. Multi-agent frameworks such as AutoGen further extend this paradigm by allowing multiple agents, humans, and tools to participate in coordinated workflows [3].

The transition from language generation to agentic execution fundamentally changes the reliability problem. In a conventional software program, an incorrect computation may produce an exception, violate a type constraint, or trigger an explicit error-handling mechanism. An LLM-based agent, however, can produce an incorrect but syntactically valid tool argument, interpret an incorrect tool response as trustworthy, select an inappropriate next action, or generate a plausible explanation for an invalid intermediate state. These failures may not trigger any conventional exception. They are therefore silent errors: faults that remain operationally invisible while contaminating subsequent computation.

This distinction is important because agentic workflows are inherently sequential. Consider an agent tasked with booking travel: User request → Intent extraction → Planning → Search → Selection → Booking → Confirmation. If the destination is incorrectly inferred during the first stage, all later steps may execute successfully from a software perspective while producing an entirely wrong outcome. The pipeline can therefore exhibit: successful execution ≠ correct execution.

Existing research reinforces this concern. AgentBench identifies poor long-horizon reasoning, decision making, and instruction following as important limitations of LLM agents [4]. SWE-bench similarly demonstrates that real-world software-engineering tasks require coordination across files, execution environments, and long contexts, making them substantially more difficult than conventional code-generation benchmarks [5]. Recent research on

agent evaluation has consequently moved toward more realistic and fine-grained assessment of agent behavior [6], [7].

A central weakness of end-to-end evaluation is that it primarily answers whether an agent succeeded, rather than where, why, and when it failed. If an agent produces an incorrect final answer after ten intermediate operations, an end-to-end score does not reveal whether the failure originated in planning, retrieval, tool selection, parameter construction, state interpretation, or final generation.

This paper therefore investigates verification checkpoints as a systematic approach to intermediate reliability. A checkpoint is defined here as an explicit verification stage inserted into an agentic workflow to determine whether an intermediate state satisfies predefined correctness, consistency, safety, or confidence conditions before execution proceeds.

The central research question is: How should verification checkpoints be designed and positioned within agentic AI pipelines to detect silent intermediate errors before they propagate into consequential downstream actions?

The paper makes four conceptual contributions: (1) It defines silent errors specifically in the context of agentic AI pipelines. (2) It develops a taxonomy of verification checkpoints according to the stage and failure mode they address. (3) It proposes risk-adaptive checkpoint placement based on error probability, impact, reversibility, and verification cost. (4) It identifies research gaps concerning checkpoint effectiveness, causal error propagation, automated verification, and standardized evaluation.

2. Background and Related Work

2.1 From language models to agentic systems

Traditional LLM applications generally follow a relatively simple pattern: $x \rightarrow f\theta(x) \rightarrow y$, where an input x is transformed into an output y .

Agentic systems introduce intermediate states and external interactions: $x \rightarrow P \rightarrow A_1 \rightarrow O_1 \rightarrow A_2 \rightarrow O_2 \rightarrow \dots \rightarrow y$, where P represents a plan, A_i an action or tool invocation, and O_i the resulting observation.

ReAct introduced a particularly influential formulation in which reasoning and actions are interleaved. The model reasons about what to do, executes an action, observes the result, and subsequently updates its reasoning [1]. This provides a foundation for modern tool-using agents, but it also creates additional opportunities for errors.

Toolformer similarly demonstrates the importance of tool invocation decisions: the model must determine whether to call an API, what arguments to supply, and how to incorporate the returned information [2]. Consequently, reliability cannot be reduced to evaluating the final generated text. Tool selection, parameters, execution results, and interpretation must also be evaluated.

AutoGen extends this principle to multi-agent workflows, where agents can communicate and coordinate with humans and tools [3]. Such architectures increase flexibility but introduce additional state transitions and communication channels at which silent errors may occur.

2.2 Reasoning and process supervision

Chain-of-thought prompting showed that explicit intermediate reasoning can improve performance on complex tasks [8]. However, exposing intermediate reasoning does not inherently guarantee that those steps are correct.

Wang et al. introduced self-consistency, which samples multiple reasoning paths and selects the most consistent answer [9]. The approach demonstrates that agreement across independent reasoning trajectories can provide a useful signal for reliability.

More directly relevant to checkpoint design, Lightman et al. compared outcome supervision with process supervision. Their results on mathematical reasoning showed that process supervision, which evaluates intermediate reasoning steps rather than only the final answer, can substantially improve reliability [10].

This distinction can be represented as: Outcome supervision: $x \rightarrow [y?]$, versus Process supervision: $x \rightarrow [s1?] \rightarrow [s2?] \rightarrow [s3?] \rightarrow [y?]$. Verification checkpoints extend this principle from model training into runtime architecture.

2.3 Hallucination and factual verification

Hallucination is one important form of silent error. An LLM may produce fluent but unsupported claims without signaling uncertainty.

SelfCheckGPT uses multiple stochastic generations to identify contradictions and inconsistencies, exploiting the observation that hallucinated information can exhibit greater variability across samples [11]. FActScore instead decomposes generated text into atomic factual claims and measures the proportion supported by reliable sources [12].

Retrieval-augmented generation systems provide another form of grounding. RAGAS evaluates retrieval-augmented systems across dimensions including retrieval quality, faithfulness to retrieved context, and generation quality [13].

These approaches suggest an important design principle: verification should operate on the specific dependency responsible for correctness. For example, if an agent's answer depends on an external database, verification should check whether the retrieved evidence supports the claim rather than merely asking another LLM whether the final answer looks correct.

2.4 Evaluation of LLM agents

AgentBench was among the early systematic benchmarks for evaluating LLMs as agents across multiple interactive environments [4]. Its results revealed substantial limitations in long-term reasoning and decision making.

Subsequent work has increasingly emphasized agent-specific evaluation. Surveys of agent evaluation distinguish capabilities such as planning and tool use from application-level performance and safety [6], [7]. This trend reflects a broader recognition that conventional NLP metrics are insufficient for autonomous workflows.

LLM-as-a-judge methods offer a scalable mechanism for evaluating open-ended outputs. Zheng et al. found that strong LLM judges can correlate reasonably well with human preferences, while also identifying biases such as position and verbosity effects [14]. Such findings are directly relevant to automated checkpoints, but they also demonstrate that a verifier is itself an imperfect model and therefore should not automatically be treated as an oracle.

3. Silent Errors in Agentic AI Pipelines

3.1 Definition

A silent error is an incorrect intermediate state, decision, or action that does not generate an explicit execution failure but can negatively affect downstream behavior.

Let an agentic pipeline contain states $S_0, S_1, \dots, S_n$ and transformations $S_{i+1} = F_i(S_i)$. A silent error occurs when $S_i \neq S_i^*$ while the execution remains operational: $\text{Exception}(F_i) = 0$. Here, S_i^* denotes the correct state.

The key distinction is therefore between: crash failures — the pipeline cannot continue; visible semantic failures — the pipeline detects an incorrect condition; and silent semantic failures — the pipeline continues despite an incorrect condition. The third category is particularly dangerous for autonomous systems.

3.2 Error propagation

Suppose each stage has an independent probability p_i of introducing an undetected error and q_i of detecting it. Even under simplified assumptions, the probability that at least one error survives across n stages increases with pipeline length: $P(\text{undetected error}) = 1 - \prod (1 - p_i(1 - q_i))$.

Real agentic systems are more complicated because errors are correlated. A wrong assumption in an early plan can systematically bias every later action.

An error can therefore exhibit propagation amplification: $E_i \rightarrow E_{i+1} \rightarrow E_{i+2} \rightarrow \dots \rightarrow E_n$. The final output may consequently contain little obvious evidence of the original fault.

3.3 Sources of silent error

Major sources include input interpretation, planning, tool selection, parameter construction, tool-output interpretation, state tracking, retrieval, reasoning, multi-agent communication, and final synthesis. The important property is that most of these failures do not necessarily produce a software exception.

4. Verification Checkpoints as a Reliability Mechanism

4.1 Conceptual model

A verification checkpoint can be represented as $C_i(S_i) \rightarrow \{\text{PASS, REPAIR, RETRY, ESCALATE, ABORT}\}$. Instead of allowing every state to proceed automatically, the system evaluates whether the state satisfies a verification predicate $V_i(S_i) = 1$. If $V_i(S_i) = 0$, the system should prevent propagation until a recovery mechanism is executed.

A checkpoint therefore transforms an agentic pipeline from $S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_3$ into $S_0 \rightarrow [C_1] \rightarrow S_1 \rightarrow [C_2] \rightarrow S_2 \rightarrow [C_3] \rightarrow S_3$.

4.2 Five-layer checkpoint architecture

Checkpoint 1: Input Verification. The first checkpoint validates the interpretation of the user's request. For example, before an agent executes "Book me the cheapest flight to Delhi tomorrow," the system should extract and verify $\{\text{destination=Delhi, date=tomorrow, objective=minimize price}\}$. For ambiguous requests, the appropriate response may be clarification rather than autonomous execution.

Checkpoint 2: Plan Verification. Before executing a multi-step plan, the agent should verify whether all required subgoals are represented, dependencies are correctly ordered, actions violate constraints, unnecessary high-risk actions exist, and the plan can be completed with available tools. A plan can be represented as $P=(a_1, a_2, \dots, a_n)$, and a verifier evaluates $VP(P, x)=1$ if P satisfies task constraints and 0 otherwise.

Checkpoint 3: Tool-Call Verification. Before invocation: $V_{pre}(\text{tool}, \text{args}, \text{context})$. After invocation: $V_{post}(\text{tool}, \text{args}, \text{result}, \text{context})$. Pre-execution verification can check tool identity, argument types, argument values, authorization, user constraints, and expected side effects. Post-execution verification can check response schema, semantic consistency, freshness, expected state changes, and correspondence between request and result.

Checkpoint 4: State Verification. Agentic systems maintain internal or external state over multiple steps. State verification checks whether the current state remains consistent with previously established facts. For state St , define invariants $I(St)=\{i_1, i_2, \dots, i_k\}$. The checkpoint verifies that every invariant remains true. Examples include user identity, budget, permissions, destination, and previously rejected options.

Checkpoint 5: Output Verification. The final checkpoint verifies the completed result against the original task, intermediate evidence, safety constraints, required format, and external ground truth where available. For factual outputs, atomic claim verification such as FActScore provides an example of fine-grained validation [12]. For retrieved information, RAGAS-style measures can assess retrieval relevance and generation faithfulness [13].

5. Designing Effective Checkpoints

5.1 Risk-adaptive checkpoint placement

Uniformly verifying every intermediate state is expensive. Instead, checkpoint placement should depend on risk. Define the risk of stage i as $R_i = P(E_i) \times I_i$, where $P(E_i)$ is the probability of an error and I_i is its potential impact. Checkpoint intensity can then be defined as $K_i = f(R_i, C_i, D_i)$, where R_i is risk, C_i verification cost, and D_i downstream propagation potential.

High-risk operations should receive stronger verification. For example, low-risk search may use result validation, a calculation may use deterministic recomputation, a database modification may use precondition and postcondition checks, and a financial transaction may require multi-factor verification and human approval. This produces a risk-adaptive architecture rather than a uniformly expensive one.

5.2 Deterministic versus model-based verification

Not all checkpoints should use another LLM. A useful hierarchy is: deterministic check > symbolic/rule check > external evidence > independent model > same-model self-evaluation.

For example, if an agent calculates $27 \times 43 = 1161$, there is little reason to ask another LLM to verify it when a calculator can provide deterministic validation. Similarly, schema validation can determine whether a JSON object is syntactically valid without requiring semantic model judgment.

LLM-based verification is most useful for tasks where deterministic verification is difficult, such as semantic consistency or natural-language interpretation.

5.3 Independent verification

A fundamental danger is correlated failure. If Agent A feeds Verifier B and both models share similar training, prompting, or reasoning weaknesses, they may independently reproduce the same error.

Self-consistency partially addresses this by sampling multiple reasoning trajectories [9]. A stronger architecture can diversify verification sources: symbolic verifier, retrieval verifier, independent LLM, and domain rule engine, with evidence aggregation. However, disagreement should not automatically be resolved by majority vote. If all verifiers rely on the same faulty evidence, agreement does not establish correctness.

6. Error Recovery and Checkpoint Actions

Detection alone is insufficient. A useful checkpoint must specify what happens when verification fails.

PASS: Continue execution when verification succeeds.

REPAIR: Modify the current intermediate state so that it satisfies required constraints.

RETRY: Repeat the failed operation under modified conditions, particularly for transient tool failures.

ROLLBACK: Restore the last verified state, $S_i \rightarrow S_{i-k}$. Rollback is valuable when an agent has performed reversible state-changing operations.

ESCALATE: Transfer the decision to a human or higher-assurance system when $P(\text{correct}) < \tau$ or the consequence of an error exceeds a defined threshold. This is especially important for high-impact actions where autonomous recovery may itself introduce additional risk.

7. Current Approaches and Developments

7.1 Process supervision

Process supervision is perhaps the closest existing research paradigm to runtime checkpointing. Lightman et al. showed that evaluating intermediate reasoning steps can outperform outcome-only supervision in mathematical reasoning [10]. Its limitation is that mathematical reasoning is considerably more structured than open-world agentic execution. A runtime agent must additionally verify external observations, tool outputs, permissions, state transitions, and environmental changes.

7.2 Self-consistency

Self-consistency generates multiple reasoning paths and selects a consistent result [9]. It can serve as a checkpoint for uncertain reasoning. However, agreement does not imply truth. If the same systematic misconception appears in all sampled trajectories, self-consistency can reinforce rather than eliminate the error. Thus, self-consistency is better viewed as an uncertainty signal than as a complete correctness guarantee.

7.3 Retrieval-grounded verification

RAG-based verification provides external evidence against which agent claims can be evaluated. RAGAS demonstrates that retrieval quality, contextual relevance, and faithfulness can be evaluated separately [13]. This suggests a checkpoint pattern: Claim $\rightarrow$ Evidence retrieval $\rightarrow$ Evidence-claim comparison $\rightarrow$ PASS/FAIL. The approach is particularly suitable for knowledge-intensive agents.

7.4 LLM-as-a-judge

LLM judges provide scalable semantic evaluation [14]. They are attractive because they can evaluate natural-language outputs without requiring task-specific deterministic rules. Nevertheless, documented biases imply that verification systems should monitor evaluator consistency, evaluator-model correlation, sensitivity to prompt changes, position bias, verbosity bias, self-preference, and calibration. A verifier should therefore itself be treated as a component requiring evaluation.

7.5 Red-team-based verification

Automated red teaming can generate adversarial cases designed to expose undesirable model behaviors [16]. For agentic systems, the same principle can be applied to checkpoints. Test cases can intentionally introduce malformed tool outputs, stale information, contradictory evidence, ambiguous instructions, missing parameters, incorrect intermediate state, prompt injection, and tool failure. The goal becomes not merely testing whether the agent completes tasks, but testing whether the verification layer detects controlled faults.

8. Proposed Verification-Checkpoint Framework

The literature suggests that no single verification technique is sufficient. This paper therefore proposes a layered architecture called VCP-AI (Verification Checkpoint Pipeline for Agentic Intelligence).

Layer 1 — Intent checkpoint: $x \rightarrow CI$. Checks task interpretation and user constraints.

Layer 2 — Planning checkpoint: $P \rightarrow CP$. Checks plan feasibility and dependency consistency.

Layer 3 — Action checkpoint: $(a, args) \rightarrow CA$. Checks tool identity, parameters, permissions, and risk.

Layer 4 — Observation checkpoint: $o \rightarrow CO$. Checks whether the returned observation is valid, relevant, fresh, and consistent.

Layer 5 — State checkpoint: $St \rightarrow CS$. Checks invariants and historical consistency.

Layer 6 — Outcome checkpoint: $y \rightarrow CY$. Checks final correctness and task satisfaction.

The overall pipeline becomes: $x \rightarrow CI \rightarrow P \rightarrow CP \rightarrow a \rightarrow CA \rightarrow o \rightarrow CO \rightarrow S \rightarrow CS \rightarrow y \rightarrow CY$. This architecture is deliberately heterogeneous: different checkpoints should use the strongest practical verification mechanism for their specific task.

8.1 Checkpoint effectiveness

A checkpoint should not simply be evaluated by final task accuracy. At least five metrics are necessary.

Detection rate: $DR = TP/(TP+FN)$ measures the fraction of introduced errors detected.

False-positive rate: $FPR = FP/(FP+TN)$ measures unnecessary interventions.

Containment rate: $CR = \text{errors prevented from reaching final action} / \text{errors introduced}$.

Verification overhead: $VO = \text{additional latency or computational cost} / \text{baseline cost}$.

Residual silent-error rate: $SER = \text{undetected harmful errors} / \text{total execution opportunities}$.

These metrics permit a meaningful comparison between checkpoint architectures.

9. Research Challenges and Limitations

9.1 The verifier can fail

The most fundamental limitation is that a verifier is itself an imperfect computational component. Agent error $\rightarrow$ Verifier $\rightarrow$ Verifier error. An LLM-based verifier may hallucinate its own justification or incorrectly approve an invalid result. Therefore, verification creates another layer of reliability engineering; it does not eliminate the reliability problem.

9.2 Correlated errors

Multiple agents or models may share the same underlying biases. Agreement among them may therefore provide less evidence than expected. Independent tools, external databases, deterministic calculations, and heterogeneous models are valuable precisely because they reduce correlated failure.

9.3 Verification cost

Every checkpoint adds latency and computation: $T_{\text{total}} = T_{\text{agent}} + \sum T_{\text{Ci}}$. For long-horizon agents, verifying every step may be prohibitively expensive. This motivates adaptive verification based on estimated risk rather than uniform checking.

9.4 Non-deterministic environments

Agentic systems frequently interact with changing environments. A web page may change, APIs may return different values, and external systems may experience transient failures. Consequently, a checkpoint may correctly identify that an output differs from a previous execution without being able to determine whether the difference is an error.

9.5 Difficulties in defining ground truth

Some agentic tasks do not have a single correct answer. Planning a trip, writing a report, or selecting a software architecture may involve multiple valid solutions. Verification must therefore distinguish incorrect from alternative but valid. This makes semantic verification substantially harder than deterministic software testing.

10. Research Gaps and Future Directions

10.1 Standardized silent-error benchmarks

Current benchmarks often emphasize end-task success. AgentBench provides multi-environment evaluation [4], while SWE-bench provides realistic software-engineering tasks [5], but neither is specifically designed around controlled injection and measurement of silent intermediate errors.

A future benchmark should intentionally inject faults at different stages: input, plan, tool, state, observation, and output, and measure whether the verification architecture detects them.

10.2 Causal error-trace datasets

Future datasets should preserve complete execution trajectories: $\tau=(S_0,A_1,O_1,\dots,A_n,O_n,S_n)$. Each trajectory should annotate first erroneous state, error type, detection point, propagation path, final consequence, and recovery outcome. Such datasets would enable research into causal fault localization, rather than merely final-answer classification.

10.3 Adaptive checkpoint scheduling

An intelligent agent should determine when additional verification is justified. A decision rule could be: Checkpoint if $P(E_i) \times I_i > \lambda C_i$, where λ represents the acceptable trade-off between risk and verification cost. This creates an optimization problem: minimize $\text{Cost}(C) + \lambda \times \text{ResidualRisk}(C)$. This formalization could lead to learned checkpoint schedulers.

10.4 Fault-injection testing for agents

Traditional software engineering relies heavily on fault injection to assess resilience. A corresponding methodology for agentic AI should intentionally introduce incorrect tool results, delayed responses, stale information, malformed arguments, contradictory observations, corrupted memory, and misleading retrieved documents. The key research question becomes: Can an agent detect that the environment is lying to it?

10.5 Verification under adversarial conditions

Verification mechanisms themselves may become attack surfaces. An adversarial document could attempt to manipulate the verifier, while prompt injection could cause the agent to bypass a checkpoint. Therefore, future architectures should treat verification as part of the security boundary rather than an independent afterthought. Recent surveys of agentic-AI security emphasize that the integration of LLMs with external tools and system components introduces security problems that differ from those of conventional software [17].

10.6 Human intervention as a calibrated resource

Human oversight should not necessarily be applied to every decision. Instead, agents could escalate only when $U(\text{autonomous action}) < U(\text{human review})$, where utility incorporates probability of error, consequence, latency, and review cost. This would transform human oversight from a generic fallback into a risk-calibrated verification mechanism.

11. Conclusion

Agentic AI systems introduce a reliability challenge that differs fundamentally from conventional text-generation systems. Their ability to reason, plan, invoke tools, maintain state, and act on external environments creates long chains of intermediate decisions. A system can therefore execute successfully while silently accumulating semantic errors.

The central argument of this paper is that verification must be distributed throughout the agentic pipeline rather than concentrated exclusively at the final output. Input interpretation, planning, tool invocation, tool observations, state transitions, and final outcomes represent distinct verification opportunities.

Existing research provides important building blocks. ReAct demonstrates the value of interleaving reasoning and acting [1]; Toolformer establishes the importance of learned tool use [2]; process supervision shows that intermediate reasoning can be more informative than final outcomes alone [10]; self-consistency provides a mechanism for assessing agreement among reasoning trajectories [9]; SelfCheckGPT and FactScore demonstrate fine-grained approaches to factual verification [11], [12]; and agent benchmarks highlight the difficulty of long-horizon autonomous behavior [4], [5].

However, these techniques do not yet constitute a unified theory of runtime error containment for agentic systems. The proposed verification-checkpoint framework addresses this gap by organizing verification around intent, planning, actions, observations, state, and outcomes, while emphasizing risk-adaptive checkpoint placement and explicit recovery actions.

Future agentic systems should therefore be evaluated not merely according to whether they complete tasks, but according to whether they can detect, localize, contain, recover from, and learn from intermediate failures. The transition from capable agents to dependable agents will depend not only on improving the underlying models, but also on engineering architectures in which incorrect reasoning and actions are difficult to propagate silently.

References

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629. DOI: 10.48550/arXiv.2210.03629.
- [2] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language Models Can Teach Themselves to Use Tools,” *arXiv preprint arXiv:2302.04761*, 2023. DOI: 10.48550/arXiv.2302.04761.
- [3] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, S. Zhang, J. Awadallah, R. Y. White, D. Burger, and C. Wang, “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation,” *arXiv preprint arXiv:2308.08155*, 2023. DOI: 10.48550/arXiv.2308.08155.
- [4] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, “AgentBench: Evaluating LLMs as Agents,” *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2308.03688. DOI: 10.48550/arXiv.2308.03688.
- [5] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770. DOI: 10.48550/arXiv.2310.06770.
- [6] M. Mohammadi, Y. Li, J. Lo, and W. Yip, “Evaluation and Benchmarking of LLM Agents: A Survey,” *arXiv preprint arXiv:2507.21504*, 2025. DOI: 10.48550/arXiv.2507.21504.
- [7] A. Yehudai, L. Edelstein, A. Li, G. Uziel, Y. Zhao, R. Bar-Haim, and M. Shmueli-Scheuer, “A Survey on Evaluation of LLM-based Agents,” *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2026.
- [8] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” *Advances in Neural Information Processing Systems (NeurIPS)*, 35, 2022, pp. 24824–24837. DOI: 10.48550/arXiv.2201.11903.
- [9] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-Consistency Improves Chain of Thought Reasoning in Language Models,” *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2203.11171. DOI: 10.48550/arXiv.2203.11171.
- [10] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, “Let’s Verify Step by Step,” *arXiv preprint arXiv:2305.20050*, 2023. DOI: 10.48550/arXiv.2305.20050.

- [11] P. Manakul, A. Liusie, and M. J. F. Gales, “SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models,” *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 9004–9017. DOI: 10.18653/v1/2023.emnlp-main.557.
- [12] S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, “FactScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation,” *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 12076–12100. DOI: 10.18653/v1/2023.emnlp-main.741.
- [13] S. Es, J. James, L. Espinosa Anke, and S. Schockaert, “RAGAs: Automated Evaluation of Retrieval Augmented Generation,” *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations (EACL)*, 2024, pp. 150–158. DOI: 10.18653/v1/2024.eacl-demo.16.
- [14] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” *arXiv preprint arXiv:2306.05685*, 2023. DOI: 10.48550/arXiv.2306.05685.
- [15] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the Middle: How Language Models Use Long Contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, 2024, pp. 157–173. DOI: 10.1162/tacl_a_00638.
- [16] E. Perez, S. Huang, F. Song, T. Cai, R. Ring, J. Aslanides, A. Glaese, N. McAleese, and G. Irving, “Red Teaming Language Models with Language Models,” *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2022. arXiv:2202.03286. DOI: 10.48550/arXiv.2202.03286.
- [17] J. Kim, X. Liu, Z. Wang, S. Qiu, B. Li, W. Guo, and D. Song, “The Attack and Defense Landscape of Agentic AI: A Comprehensive Survey,” *arXiv preprint arXiv:2603.11088*, 2026. DOI: 10.48550/arXiv.2603.11088.